AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Algorytm listy dwukierunkowej z zastosowaniem GitHub

Autor:
Jelito Tomasz

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

1.1. Cele i założenia projektu 3

1.2. Środowisko technologiczne 3

2. Analiza problemu **5**

2.1. Charakterystyka listy dwukierunkowej 5

2.2. Analiza wzorców projektowych 5

2.3. System kontroli wersji Git oraz GitHub 6

3. Projektowanie **7**

3.1. Środowisko i narzędzia programistyczne 7

3.2. Architektura systemu i diagram klas 7

3.3. Wykorzystanie systemu kontroli wersji 7

4. Implementacja **9**

4.1. Struktura kodu źródłowego 9

4.2. Kluczowe fragmenty kodu i pliki źródłowe 9

4.3. Wyniki działania programu i kontrola wersji 16

4.4. Dokumentacja operacji w systemie Git 16

5. Wnioski **20**

5.1. Podsumowanie techniczne 20

5.2. Aspekty inżynieryjne i narzędziowe 20

Literatura **21**

Spis rysunków **21**

Spis tabel **22**

Spis listingów **23**

1. Ogólne określenie wymagań

Głównym celem opisywanego projektu jest stworzenie, zaprogramowanie i opracowanie dokumentacji dla dynamicznej struktury danych, jaką jest lista dwukierunkowa. Aplikację napisano w języku C++, opierając się na uznanych wzorcach projektowych i stosując współczesne praktyki inżynierii oprogramowania.

1.1. Cele i założenia projektu

Poniżej przedstawiono kluczowe wytyczne technologiczne i funkcjonalne, które musiał spełnić realizowany program:

- **Struktura danych:** Zaprogramowanie mechanizmu listy dwukierunkowej (obejmujące obiekty węzłów, główną klasę zarządzającą oraz logikę niezbędnych operacji).
- **Wzorce projektowe:**
  - *Iterator* – pozwalający na bezpieczne przechodzenie po elementach kolekcji bez ujawniania jej wewnętrznej budowy.
  - *Fabryka (ListFactory)* – odpowiedzialna za separację i ścisłą kontrolę nad procesem instancjonowania obiektów nowej listy.
- **Modularność kodu:** Rozdzielenie deklaracji od implementacji poprzez logiczny podział na pliki nagłówkowe (`.h`) oraz kod źródłowy (`.cpp`) dla każdego z komponentów.
- **Kontrola wersji:** Zastosowanie systemu Git połączonego z serwisem GitHub w celu śledzenia postępów prac (m.in. zatwierdzanie zmian, wycofywanie commitów, klonowanie czy obsługa zdalnego repozytorium).
- **Dokumentacja techniczna:** Automatyczne wygenerowanie opisu technicznego z komentarzy umieszczonych w kodzie, wykorzystując narzędzie Doxygen z opcją eksportu do formatu LaTeX.

1.2. Środowisko technologiczne

Prace nad projektem, w tym pisanie kodu i jego weryfikacja, odbywały się przy użyciu poniższego zestawu oprogramowania:

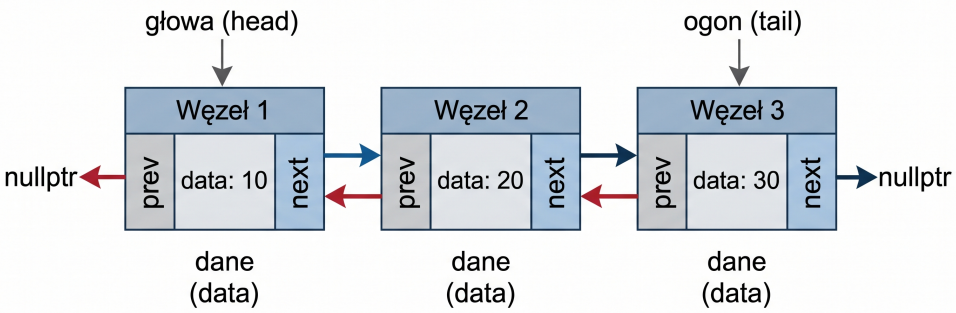
- **Język programowania:** C++.

- **Środowisko programistyczne (IDE):** Visual Studio.
- **Kompilator:** MSYS2 / MinGW (GCC / G++).
- **Kontrola wersji:** Git.
- **Generator dokumentacji:** Doxygen.

2. Analiza problemu

2.1. Charakterystyka listy dwukierunkowej

Lista dwukierunkowa to zaawansowana, liniowa struktura danych. Jej specyfika opiera się na tym, że każdy węzeł (pojedynczy element) przechowuje obok własnej wartości również dwa wskaźniki adresujące – jeden na element poprzedzający, a drugi na następujący. W odróżnieniu od standardowych tablic dynamicznych, taka konstrukcja pozwala na błyskawiczne wstawianie i usuwanie elementów w dowolnym punkcie struktury w czasie stałym $O(1)$, pod warunkiem posiadania referencji do danego węzła. Rozwiązanie to jest niezwykle użyteczne w programach wymagających ciągłych modyfikacji danych, na przykład w implementacji mechanizmów cofania akcji (tzw. *undo/redo*) lub w złożonych kolejkach operacji. Schemat budowy i powiązań wewnątrz takiej struktury zilustrowano na rysunku 2.1.



Rys. 2.1. Schemat logiczny powiązań w liście dwukierunkowej (węzły, wartości oraz wskaźniki `prev` i `next`)

2.2. Analiza wzorców projektowych

W architekturze projektu zaimplementowano dwa kluczowe wzorce projektowe:

- **Iterator:** Należący do grupy wzorców behawioralnych, pozwala na bezpieczne i sekwencyjne przechodzenie przez wszystkie elementy kolekcji, całkowicie ukrywając jej wewnętrzną budowę przed użytkownikiem. Udostępnia on ujednolicony interfejs służący do poruszania się w obu kierunkach (w przód i w tył) oraz do odczytywania wartości węzłów.

- **Fabryka (ListFactory):** Reprezentuje kreacyjny wzorzec projektowy, którego celem jest całkowite zhermetyzowanie procesu powoływania do życia nowych obiektów. Dzięki niemu fizyczna alokacja pamięci dla nowej listy zostaje wyraźnie oddzielona od głównej logiki biznesowej programu.

2.3. System kontroli wersji Git oraz GitHub

Aby zachować pełną kontrolę nad ewolucją kodu źródłowego, wykorzystano rozproszony system Git w połączeniu z platformą hostingową GitHub. Taki zestaw narzędzi pozwala na precyzyjne zapisywanie poszczególnych faz projektu (poprzez system commitów), dbanie o spójność plików oraz wersjonowanie tworzonej dokumentacji. Ponadto, dzięki funkcjom klonowania, tworzenia odgałęzień i wysyłania zmian na zdalny serwer, środowisko to idealnie symuluje warunki profesjonalnej pracy zespołowej.

3. Projektowanie

3.1. Środowisko i narzędzia programistyczne

Aplikacja została napisana w języku C++ z wykorzystaniem środowiska programistycznego Visual Studio. Kod źródłowy kompilowano przy użyciu kompilatora GCC/G++ wchodzącego w skład pakietu MSYS2 (profil UCRT64). Do archiwizacji i nadzorowania postępów prac wykorzystano system kontroli wersji Git, współpracujący z chmurową platformą GitHub.

3.2. Architektura systemu i diagram klas

Budowa aplikacji opiera się na obiektowym i modularnym podejściu, dzieląc logikę na deklaracje w plikach nagłówkowych (`.h`) oraz implementacje w plikach źródłowych (`.cpp`). W obrębie projektu zdefiniowano następujące kluczowe moduły:

- **Klasa `Node`:** Pełni funkcję pojedynczego ogniwa (węzła) listy, przechowując informacje o konkretnej wartości oraz adresy pamięci swoich bezpośrednich sąsiadów (wstecz i w przód).
- **Klasa `DoublyLinkedList`:** Stanowi główny silnik zarządzający całą kolekcją. Zapewnia kompleksowy zestaw metod służących do dodawania, usuwania i wyświetlania zawartości struktury.
- **Klasa `ListIterator`:** Wdraża założenia behawioralnego wzorca Iterator, oferując hermetyczny i bezpieczny interfejs do płynnej nawigacji pomiędzy zgromadzonymi elementami.
- **Klasa `ListFactory`:** Reprezentuje kreacyjny wzorzec Fabryki, przejmując na siebie wyłączną odpowiedzialność za prawidłowe i bezpieczne instancjonowanie nowych obiektów listy.

3.3. Wykorzystanie systemu kontroli wersji

Przebieg tworzenia oprogramowania był na bieżąco archiwizowany i koordynowany za pośrednictwem narzędzia Git. Podczas cyklu deweloperskiego wykonano następujące kroki:

- Utworzenie i inicjalizację lokalnego środowiska repozytorium dla plików projektu.

- Dokumentowanie kolejnych kamieni milowych implementacji poprzez systematyczne tworzenie punktów przywracania (wygenerowanie zestawu minimum 5 oddzielnych commitów).
- Zestawienie połączenia między komputerem lokalnym a zdalnym serwerem GitHub, co pozwoliło na wypychanie wprowadzonych zmian na zewnątrz przy użyciu polecenia `git push`.

4. Implementacja

4.1. Struktura kodu źródłowego

Kod aplikacji powstał w języku C++ w oparciu o silnie zarysowaną architekturę modularną. Każdy istotny element logiczny systemu rozdzielono na pliki nagłówkowe z definicjami (`.h`) oraz pliki z właściwym kodem wykonawczym (`.cpp`). W skład oprogramowania wchodzą cztery główne filary: bazowy element danych (`Node`), klasa nadrzędna sterująca całą strukturą (`DoublyLinkedList`), dedykowany mechanizm nawigacyjny (`ListIterator`) oraz klasa kreacyjna służąca bezpiecznej alokacji (`ListFactory`).

4.2. Kluczowe fragmenty kodu i pliki źródłowe

Prawidłowe działanie listy dwukierunkowej opiera się na precyzyjnym operowaniu wskaźnikami oraz izolacji krytycznych operacji przy pomocy sprawdzonych wzorców projektowych.

Kod 1 (s. 9) ukazuje konstrukcję pojedynczego węzła (`Node.cpp`), w którym inicjalizowana jest wartość liczbowa oraz bezpiecznie zerowane są wskaźniki kierunkowe. Z kolei kod 2 (s. 10) ilustruje hermetyczną logikę obsługi iteratora ukrytą w pliku `ListIterator.cpp`. Jego działanie polega nie tylko na samym przemieszczaniu wskaźnika (w przód i w tył), ale również na bezpośrednim wypisywaniu zawartości odwiedzanego węzła na ekran konsoli za pomocą wbudowanych metod takich jak `showNext()` czy `showPrev()`.

Za integralność danych i logikę samej listy odpowiada kod 3 (s. 11) zawarty w pliku `DoublyLinkedList.cpp`. Realizuje on pełen zestaw metod modyfikujących kolekcję (m.in. `pushFront`, `popBack`, `insertAt`) i na bieżąco zarządza skrajnymi wskaźnikami `head` oraz `tail`.

Aby zrealizować wymóg działania struktury na stercie pamięciowej, zastosowano wzorzec zaprezentowany w kodzie 4 (s. 14). Plik `ListFactory.cpp` deleguje proces instancjonowania obiektu do oddzielnej klasy, ukrywając przed użytkownikiem wywołanie operatora `new`.

Zwieńczeniem architektury jest plik `main.cpp` (kod 5, s. 14), w którym przeprowadzono rygorystyczne testy działania logiki. Obrazuje on pełen cykl życia oprogramowania: od wywołania fabryki, przez modyfikację węzłów i testowanie nawigacji iteratora, aż po finalne zwolnienie zasobów pamięci za pomocą instrukcji `delete`.

```
1 #include "Node.h"
```

```
2
3  Node::Node(int val) {
4      data = val;
5      prev = nullptr;
6      next = nullptr;
7  }
```

Listing 1. Implementacja węzła listy dwukierunkowej

```
1  #include "ListIterator.h"
2  #include <iostream>
3
4  ListIterator::ListIterator(Node* start) {
5      current = start;
6  }
7
8  void ListIterator::showNext() {
9      if (current != nullptr && current->next != nullptr) {
10         current = current->next;
11         std::cout << "Nastepny element: " << current->data << std::
       endl;
12     }
13     else {
14         std::cout << "Brak nastepnego elementu (koniec listy)." <<
       std::endl;
15     }
16 }
17
18 void ListIterator::showPrev() {
19     if (current != nullptr && current->prev != nullptr) {
20         current = current->prev;
21         std::cout << "Poprzedni element: " << current->data << std
       ::endl;
22     }
23     else {
24         std::cout << "Brak poprzedniego elementu (poczatek listy)."
       << std::endl;
25     }
26 }
27
28 void ListIterator::showCurrent() const {
29     if (current != nullptr) {
30         std::cout << "Obecny element: " << current->data << std::
       endl;
31     }
32     else {
```

```
33          std::cout << "Iterator nie wskazuje na zaden element." <<
    std::endl;
34      }
35  }
36
37  bool ListIterator::isValid() const {
38      return current != nullptr;
39  }
```

Listing 2. Implementacja mechanizmu nawigacji (ListIterator)

```
1   #include "DoublyLinkedList.h"
2   #include <iostream>
3
4   DoublyLinkedList::DoublyLinkedList() {
5       head = nullptr;
6       tail = nullptr;
7       size = 0;
8   }
9
10  DoublyLinkedList::~DoublyLinkedList() {
11      clear();
12  }
13
14  void DoublyLinkedList::pushFront(int val) {
15      Node* newNode = new Node(val);
16      if (head == nullptr) {
17          head = tail = newNode;
18      }
19      else {
20          newNode->next = head;
21          head->prev = newNode;
22          head = newNode;
23      }
24      size++;
25  }
26
27  void DoublyLinkedList::pushBack(int val) {
28      Node* newNode = new Node(val);
29      if (tail == nullptr) {
30          head = tail = newNode;
31      }
32      else {
33          tail->next = newNode;
34          newNode->prev = tail;
35          tail = newNode;
```

```
36      }
37      size++;
38  }
39
40  void DoublyLinkedList::insertAt(int index, int val) {
41      if (index <= 0) {
42          pushFront(val);
43          return;
44      }
45      if (index >= size) {
46          pushBack(val);
47          return;
48      }
49
50      Node* current = head;
51      for (int i = 0; i < index; i++) {
52          current = current->next;
53      }
54
55      Node* newNode = new Node(val);
56      newNode->prev = current->prev;
57      newNode->next = current;
58      current->prev->next = newNode;
59      current->prev = newNode;
60      size++;
61  }
62
63  void DoublyLinkedList::popFront() {
64      if (head == nullptr) return;
65
66      Node* temp = head;
67      head = head->next;
68
69      if (head != nullptr) {
70          head->prev = nullptr;
71      }
72      else {
73          tail = nullptr; // Lista stala sie pusta
74      }
75
76      delete temp;
77      size--;
78  }
79
80  void DoublyLinkedList::popBack() {
```

```
81      if (tail == nullptr) return;
82
83      Node* temp = tail;
84      tail = tail->prev;
85
86      if (tail != nullptr) {
87          tail->next = nullptr;
88      }
89      else {
90          head = nullptr; // Lista stala sie pusta
91      }
92
93      delete temp;
94      size--;
95  }
96
97  void DoublyLinkedList::removeAt(int index) {
98      if (index < 0 || index >= size || head == nullptr) return;
99
100     if (index == 0) {
101         popFront();
102         return;
103     }
104     if (index == size - 1) {
105         popBack();
106         return;
107     }
108
109     Node* current = head;
110     for (int i = 0; i < index; i++) {
111         current = current->next;
112     }
113
114     current->prev->next = current->next;
115     current->next->prev = current->prev;
116
117     delete current;
118     size--;
119 }
120
121 void DoublyLinkedList::displayForward() const {
122     Node* current = head;
123     std::cout << "Lista (od poczatku): ";
124     if (current == nullptr) {
125         std::cout << "pusta";
```

```
126     }
127     while (current != nullptr) {
128         std::cout << current->data << " ";
129         current = current->next;
130     }
131     std::cout << std::endl;
132 }
133 
134 void DoublyLinkedList::displayBackward() const {
135     Node* current = tail;
136     std::cout << "Lista (od konca): ";
137     if (current == nullptr) {
138         std::cout << "pusta";
139     }
140     while (current != nullptr) {
141         std::cout << current->data << " ";
142         current = current->prev;
143     }
144     std::cout << std::endl;
145 }
146 
147 void DoublyLinkedList::clear() {
148     while (head != nullptr) {
149         popFront();
150     }
151 }
152 
153 ListIterator DoublyLinkedList::getIterator() const {
154     return ListIterator(head);
155 }
```

Listing 3. Implementacja głównej klasy zarządzającej listą

```
1 #include "ListFactory.h"
2 
3 DoublyLinkedList* ListFactory::createList() {
4     // Alokacja obiektu listy w pamieci sterty (heap) za pomoca
    operatora new
5     return new DoublyLinkedList();
6 }
```

Listing 4. Wzorzec kreacyjny Factory ułatwiający alokację

```
1 #include <iostream>
2 #include "ListFactory.h"
3 #include "DoublyLinkedList.h"
```

```cpp
#include "ListIterator.h"

using namespace std;

int main() {
    cout << "--- Inicjalizacja poprzez wzorzec Factory ---" << endl;
    DoublyLinkedList* myList = ListFactory::createList();

    cout << "\n--- TEST: Dodawanie elementow ---" << endl;
    myList->pushFront(20);    // Dodaj na poczatek
    myList->pushFront(10);    // Dodaj na poczatek
    myList->pushBack(40);     // Dodaj na koniec
    myList->pushBack(50);     // Dodaj na koniec
    myList->insertAt(2, 30); // Dodaj 30 pod indeks 2 (powinno byc miedzy 20 a 40)

    myList->displayForward();  // Wymog: Wyswietl cala liste
    myList->displayBackward(); // Wymog: Wyswietl liste w odwrotnej kolejnosci

    cout << "\n--- TEST: Wzorzec Iterator ---" << endl;
    ListIterator it = myList->getIterator();
    if (it.isValid()) {
        it.showCurrent();
        it.showNext(); // Wymog: Wyswietl nastepny element
        it.showNext();
        it.showPrev(); // Wymog: Wyswietl poprzedni element
    }

    cout << "\n--- TEST: Usuwanie elementow ---" << endl;
    cout << "Usuwam z poczatku i konca:" << endl;
    myList->popFront(); // Usuwa 10
    myList->popBack();  // Usuwa 50
    myList->displayForward();

    cout << "Usuwam z pod wskazanego indeksu (indeks 1, czyli liczba 40):" << endl;
    myList->removeAt(1);
    myList->displayForward();

    cout << "\n--- TEST: Czyszczenie listy ---" << endl;
    myList->clear(); // Wymog: Czysc cala liste
    myList->displayForward();

```

```
45      // Wymog: Klasa dziala na stercie, wiec po tescie czyscimy
        pamiec
46      delete myList;
47      cout << "\nPamiec zwolniona, program zakonczony." << endl;
48
49      return 0;
50      //Test
51  }
```

Listing 5. Główny plik testowy programu obrazujący wykorzystanie klas

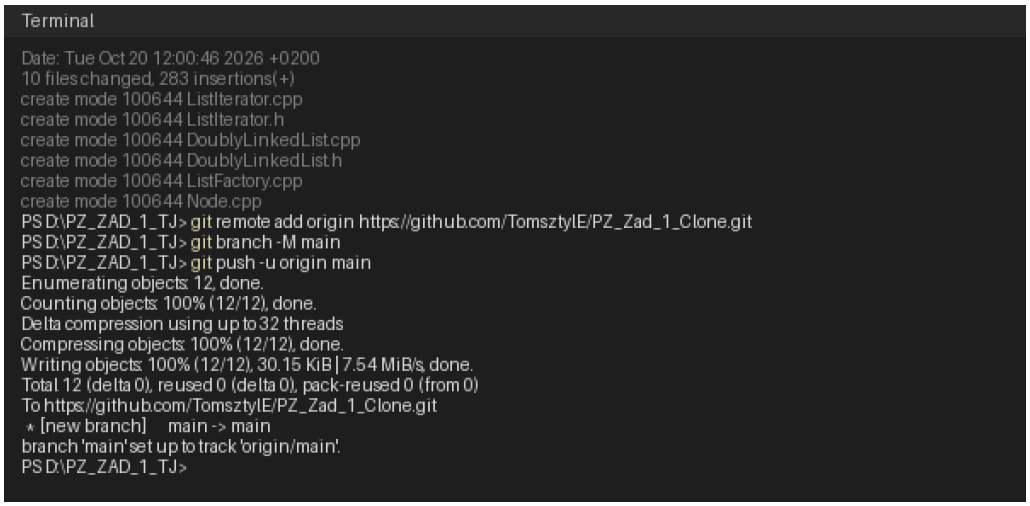
4.3. Wyniki działania programu i kontrola wersji

Kompilacja i uruchomienie programu w pełni potwierdza poprawne funkcjonowanie zaimplementowanej logiki: aplikacja bezbłędnie alokuje instancję nowej listy poprzez fabrykę, wstawia węzły i je modyfikuje, a na końcu pozwala na płynne iterowanie w obu kierunkach z równoczesnym wyświetlaniem danych.

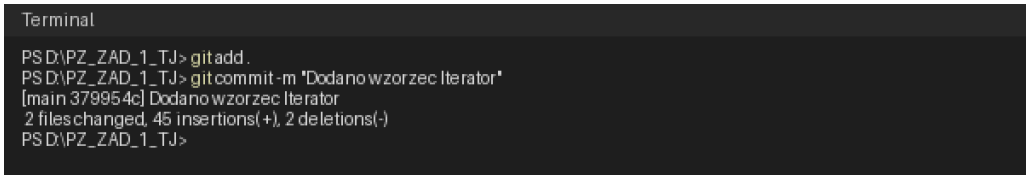
Równolegle z powstawaniem kodu w C++, prowadzono systematyczny zapis historii projektu za pomocą oprogramowania Git. Powstała w ten sposób stabilna sekwencja commitów, którą po zakończeniu najważniejszych etapów prac zsynchronizowano z zewnętrzną platformą GitHub.

4.4. Dokumentacja operacji w systemie Git

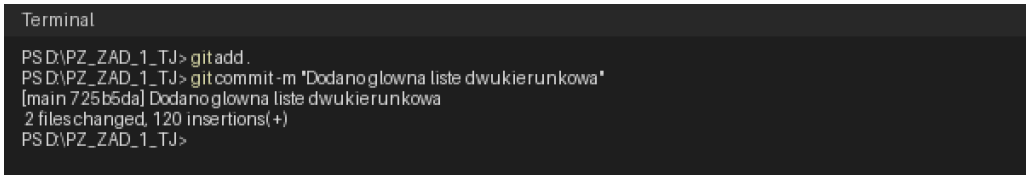
Zwieńczeniem prac nad kodem było przetestowanie w praktyce szeregu zaawansowanych komend kontroli wersji. Poniższe ilustracje dokumentują przebieg takich operacji jak: budowa systematycznej historii, cofanie stanu projektu, symulacja pracy na dwóch oddzielnych maszynach oraz odtwarzanie utraconych plików źródłowych z repozytorium zdalnego.



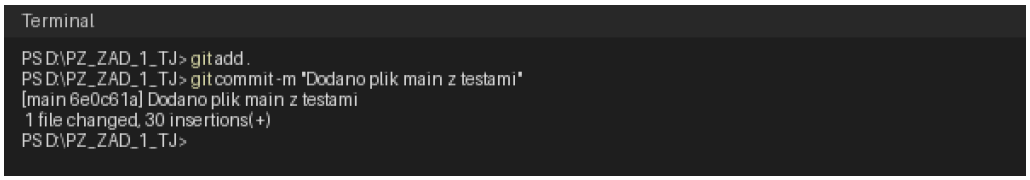
Rys. 4.1. Inicjalizacja lokalnego repozytorium oraz przesłanie danych na serwer (pierwszy push)



Rys. 4.2. Zatwierdzenie drugiego commita w historii projektu



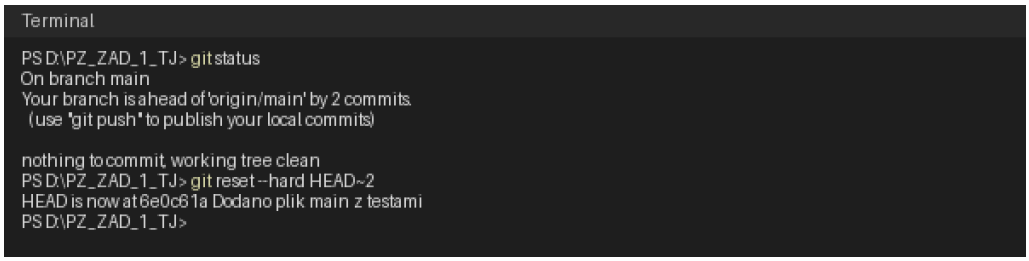
Rys. 4.3. Trzeci commit ilustrujący dodanie pierwszych komentarzy w kodzie



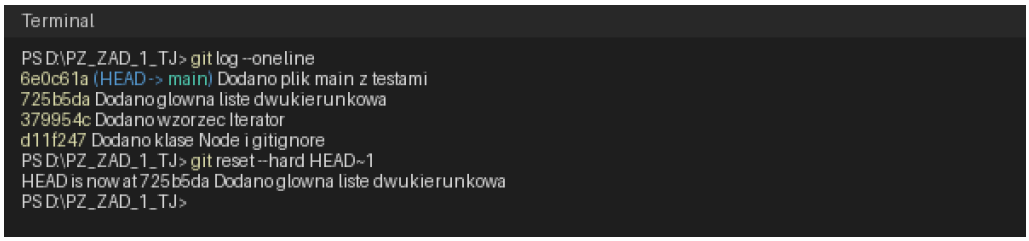
Rys. 4.4. Czwarty zapis stanu prac (commit)



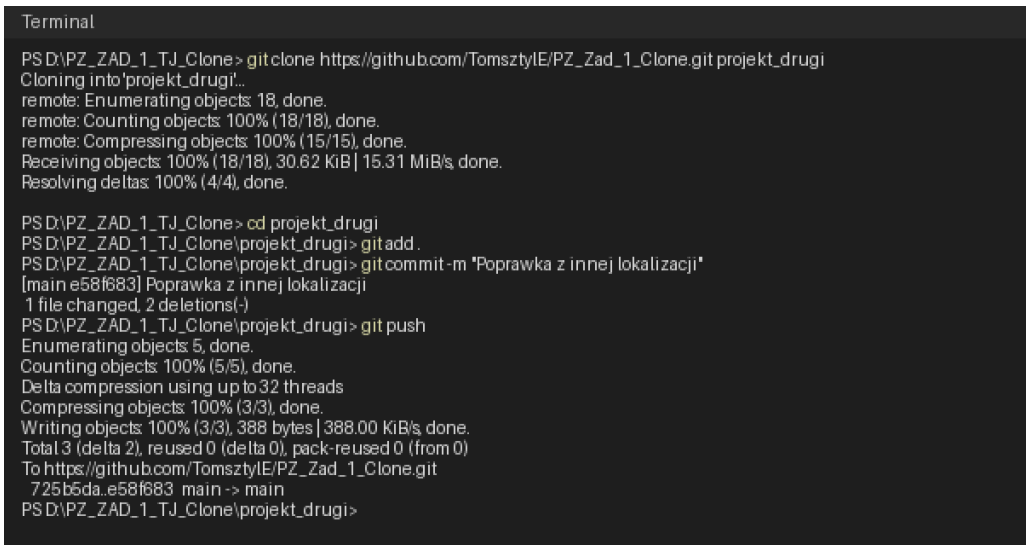
Rys. 4.5. Piąty commit, stanowiący punkt wyjścia do zaawansowanych modyfikacji historii



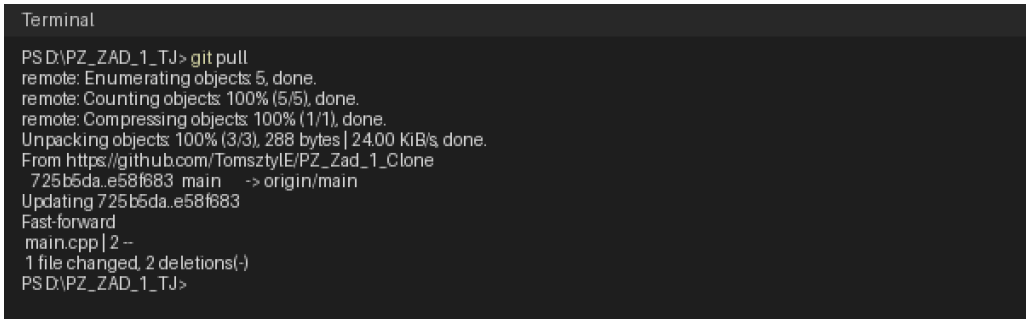
Rys. 4.6. Cofnięcie wskaźnika HEAD o dwa commity wstecz za pomocą polecenia `git reset --hard HEAD~2`



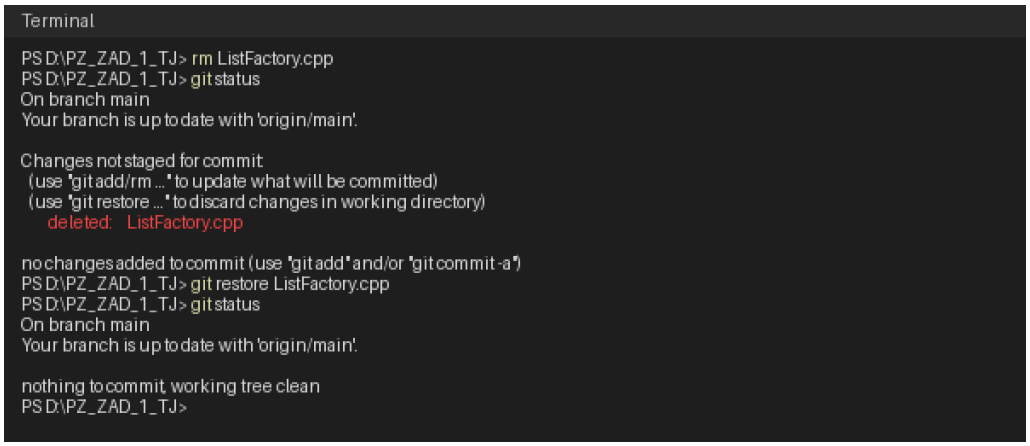
Rys. 4.7. Weryfikacja operacji rebase i skuteczne usunięcie pojedynczego zapisu z wierzchołka historii (`git reset --hard HEAD~1`)



Rys. 4.8. Klonowanie repozytorium do drugiego środowiska (`projekt_drugi`), edycja plików powiązana z symulacją i wysłanie zmian (`git push`)



Rys. 4.9. Pobranie zaktualizowanych zmian z powrotem do pierwszej lokalizacji przy użyciu polecenia `git pull`



Rys. 4.10. Symulacja ludzkiego błędu – usunięcie ważnego pliku źródłowego i jego bezproblemowe odzyskanie komendą `git restore`

5. Wnioski

Przeprowadzone prace programistyczne pozwoliły na pełne zrealizowanie postawionych celów projektowych. W rezultacie dostarczono działającą i wolną od błędów implementację listy dwukierunkowej w języku C++, wspartą sprawdzonymi w branży wzorcami projektowymi, takimi jak Iterator oraz Fabryka (Factory).

5.1. Podsumowanie techniczne

Skrupulatny podział projektu na niezależne moduły, odzwierciedlony w separacji plików nagłówkowych (`.h`) i implementacyjnych (`.cpp`), znacząco podniósł czytelność, elastyczność oraz łatwość w utrzymaniu całego rozwiązania. Wdrożenie wzorca Iterator zagwarantowało użytkownikowi bezpieczne przemieszczanie się po strukturze danych, chroniąc jednocześnie wewnętrzne wskaźniki węzłów przed niepożądaną, bezpośrednią modyfikacją. Równolegle, zastosowanie klasy `ListFactory` pozwoliło wyabstrahować logikę zarządzania pamięcią operacyjną, skutecznie oddzielając proces tworzenia i alokacji obiektów od głównego rdzenia (logiki biznesowej) programu.

5.2. Aspekty inżynieryjne i narzędziowe

Konsekwentne stosowanie systemu Git we współpracy z chmurowym serwisem GitHub zapewniło pełną kontrolę nad procesem deweloperskim. Środowisko to umożliwiło bezpieczne archiwizowanie kolejnych wersji kodu za pomocą commitów, testowanie zaawansowanych operacji na historii oraz bezproblemowe zarządzanie repozytorium. Dopełnieniem całości prac inżynierskich było zintegrowanie kodu źródłowego z generatorem Doxygen. Wykorzystanie tego narzędzia do zautomatyzowanego wyciągania informacji i ich eksportu do środowiska LaTeX zaowocowało niezwykle spójną, estetyczną i profesjonalną dokumentacją techniczną, która w pełni odpowiada rygorystycznym standardom akademickim.

Spis rysunków

2.1. Schemat logiczny powiązań w liście dwukierunkowej (węzły, wartości oraz wskaźniki `prev` i `next`) 5
4.1. Inicjalizacja lokalnego repozytorium oraz przesłanie danych na serwer (pierwszy push) 17
4.2. Zatwierdzenie drugiego commita w historii projektu 17
4.3. Trzeci commit ilustrujący dodanie pierwszych komentarzy w kodzie . 17
4.4. Czwarty zapis stanu prac (commit) 17
4.5. Piąty commit, stanowiący punkt wyjścia do zaawansowanych modyfikacji historii 17
4.6. Cofnięcie wskaźnika HEAD o dwa commity wstecz za pomocą polecenia `git reset --hard HEAD~2` 18
4.7. Weryfikacja operacji rebase i skuteczne usunięcie pojedynczego zapisu z wierzchołka historii (`git reset --hard HEAD~1`) 18
4.8. Klonowanie repozytorium do drugiego środowiska (`projekt_drugi`), edycja plików powiązana z symulacją i wysłanie zmian (`git push`) . . 18
4.9. Pobranie zaktualizowanych zmian z powrotem do pierwszej lokalizacji przy użyciu polecenia `git pull` 19
4.10. Symulacja ludzkiego błędu – usunięcie ważnego pliku źródłowego i jego bezproblemowe odzyskanie komendą `git restore` 19

Spis tabel

Spis listingów

1. Implementacja węzła listy dwukierunkowej 9
2. Implementacja mechanizmu nawigacji (ListIterator) 10
3. Implementacja głównej klasy zarządzającej listą 11
4. Wzorzec kreacyjny Factory ułatwiający alokację 14
5. Główny plik testowy programu obrazujący wykorzystanie klas 14